

Gem Cascade

پیاده‌سازی این پروژه شامل طراحی بخش‌های منطقی و پیاده‌سازی بخش‌های نمایشی آن بود؛ می‌توان گفت که پیاده‌سازی اصولی جدول بازی و منطق‌های مربوط به تطابق‌ها، مهم‌ترین گام در این پروژه بود.

-چالش‌ها:

اولین چالش در این پروژه ایجاد خطوط جدول برای نمایش آن است. من در ابتدا از کاراکترهای "+-" برای ساخت جدول استفاده کردم و بعد از اتمام نسخه‌ی اول پروژه، با استفاده از کاراکترهای یونیکد ظاهر زیباتری به جدول و کنترلر پروژه دادم.

چالش بعدی انتخاب کاراکترها برای پروژه بود؛ ابتدا از 5 حرف اول حروف الفبای انگلیسی استفاده کردم که این انتخاب باعث ایجاد چالش بعدی شد. با توجه به اینکه تعریف تطابق در پروژه ما با تعریف تطابق در نسخه‌های مشابه این بازی تفاوت اساسی دارد، بعد از ایجاد یک تطابق، تعداد زیادی تطابق پشت سرهم در اثر جایگزین شدن جواهرات از بین رفته ایجاد می‌شد و امتیاز اضافی زیادی تولید می‌کرد. بنابراین تعداد کاراکترها را به 8 تا افزایش دادم و بعد از ایجاد تغییرات بر روی نسخه‌ی اول پروژه، کاراکترها را از حروف الفبا به ایموجی‌های میوه تغییر دادم تا ظاهر زیباتری پیدا کند.

تولید تصادفی کاراکتر برای پرکردن جدول هم از چالش‌های اصلی این پروژه بود. با جستجو در اینترنت نحوه انتخاب یک عدد تصادفی در یک بازه از اعداد را پیدا کردم و بعد از آن تابع تولید کاراکتر تصادفی را در پروژه پیاده سازی کردم؛ البته راه حل اولیه‌ای که پیدا کردم ظاهراً منسوخ شده بود و به اندازه کافی به صورت رندوم کار نمی‌کرد و باعث ایجاد حلقه‌های نامتناهی در ساختار منطقی کد می‌شد.

چک کردن خانه‌های جدول و پیدا کردن تطابق‌های n -تایی بزرگترین چالش این پروژه بود که با توجه به تعریف تطابق در ساختار این پروژه، نیاز به استفاده از الگوریتم DFS داشتم؛ از این الگوریتم در بسیاری از بخش‌های پروژه استفاده می‌شود. تابع DFS در این پروژه سه پارامتر ورودی دارد که به ترتیب شماره سطر، شماره ستون و یک وکتور از خانه‌هایی که با هم تطابق دارند، هستند. در ابتدا این تابع را با دو ورودی و البته با یک خروجی که همان وکتور بود طراحی کرده بودم ولی برای سریع‌تر شدن و مصرف بهینه‌تر حافظه، با استفاده از Reference Variable و Pointer این تابع را به شکل نهایی و بهینه‌تر تبدیل کردم.

برای نمایش بهتر gameplay بازی و منو های مربوطه از Sleep و cls استفاده کردم تا بعد از هر حرکت یک زمان مشخصی جدول نمایش داده شود و ترمینال پاکسازی شود تا برای نمایش دوباره آماده شود. برای رنگی نمایش دادن منو ها و کاراکتر های جدول (در نسخه ابتدایی) نیز از رنگ ها ASCII با ایجاد define های مختلف با اسم همان رنگ استفاده کردم.

برای افزودن قابلیت های کمکی راکت و بمب و راهنمایی نیز از همان توابع قبلی استفاده کردم تا ساختار کد تمیز بماند و از تکرار جلوگیری شود. قابلیت های بمب و راکت ابتدا خانه های تحت تاثیر خود را نابود میکنند و بعد از آن توابع مربوط به شمارش امتیاز و اعمال جاذبه و ساخت جواهرات جدید را فراخوانی میکنند. قابلیت راهنمایی نیز تمامی حالات ممکن برای جا به جایی را بررسی می کند و آن حالتی که بیشترین تعداد تطابق را می سازد معرفی میکند؛ در این بررسی از توابع قبلی که برای بررسی جدول بعد از جا به جایی دو خانه توسط کاربر تعریف شده بود، استفاده می شود.

برای ایجاد یک نقطه ی پایان برای بازی تصمیم گرفتم برای آن سه سطح مختلف با توجه به شیوه امتیازدهی طراحی کنم که در هر سطح یک امتیاز هدف داریم و تعداد محدودی حرکت که بایستی تا قبل از اتمام تعداد حرکات به امتیاز هدف برسیم در غیر این صورت بازنده خواهیم بود.

برای اینکه بازی قابلیت ذخیره و بارگذاری آخرین بازی نیمه کاره را داشته باشد از ifstream ، fstream و ofstream استفاده کردم تا اطلاعات ضروری برای پردازش بازی را در فایل متنی با پسوند txt ذخیره کنم و از روی آن بازخوانی کنم. جهت ایجاد سهولت در این امر برای هر کدام از ایموهای های میوه ای که در جدول بازی نشان داده میشوند یک حرف از الفبای زبان انگلیسی انتخاب کردم تا به جای ایموهای ها، آن ها را در فایل ذخیره کنم و از روی فایل بخونم.

بعد از اتمام اولین ورژن پروژه، در ورژن دوم ابتدا هر بخش را در کلاس جداگانه ای نوشتم تا با استفاده از SRP ساختار کد را تمیز تر و خوانا تر کنم و افزودن قابلیت جدید به آن را ساده تر کنم تا توسعه بازی راحت تر باشد (OCP). سپس هر کدام از کلاس ها را در فایل جداگانه ای قرار دادم تا طراحی پروژه ماژولار و مرتب تر شود.

یک فایل حاوی توضیحاتی در مورد بازی و شیوه انجام آن هم طراحی کردم که با انتخاب آن در منوی اولیه میتوان فایل را مطالعه کرد تا از شیوه درست انجام بازی مطلع شد. همچنین در مقابل هر یک از امکانات بازی در کنترلر توضیحات مختصری در مورد شیوه استفاده از آن قابلیت را قرار دادم. در پیاده سازی این پروژه سعی

کردم که علاوه بر استفاده از شی‌گرایی، از اصول SOLID هم تا حد امکان پیروی کنم و برنامه ساختار تمیز و خوانایی داشته باشد.

-منابع:

- [create random number](#)
- [Writing on files in cpp](#)
- [ASCII color code](#)

```
Enter number of your choice:
1.New Game
2.Load Game
3.About this game
4.Exit
|
```

```
Choose the game level:
1.Easy
2.Medium
3.Hard
4.Back
|
```



تصاویری از محیط بازی

جهت دانلود فایل نصبی پروژه رو این [لینک](#) کلیک کنید.

دسترسی به ریپازیتوری از این [لینک](#).